ROADMAP DE DESENVOLVIMENTO · V1.1

MangaShelf

Plataforma open source de hospedagem de mangás, robusta e fácil de auto-hospedar.

Documento técnico que descreve visão, arquitetura, stack, sistema de permissões atômicas, modelo VIP, painel administrativo, requisitos de segurança OWASP, armazenamento compatível com S3 e o roadmap de entrega em fases do projeto MangaShelf.

GO + NEXT.JS | POSTGRESQL | AGPL-3.0 | SELF-HOSTED

STACK
Go · Next.js · Docker

LICENÇA
AGPL-3.0

DOCUMENTO TÉCNICO

VERSÃO 1.1

Sumário

1. Visão e princípios **5**

2. Stack e decisões **5**

3. Arquitetura e sistema de flags **6**

3.1 Flags de configuração 6

3.2 Interfaces (portas) 6

3.3 Regras importantes 7

3.4 Módulos internos do Go 7

3.5 Fluxo de upload 7

3.6 Fluxo de leitura 8

3.7 Compatibilidade com app mobile 8

4. Autenticação e autorização **10**

4.1 Autenticação 10

4.2 Autorização por permissões atômicas 10

5. VIP e controle de acesso ao conteúdo **11**

5.1 Regras definidas pelo publicador 11

5.2 Como funciona 11

5.3 Proteção das imagens 12

5.4 Modelo de dados 12

6. Painel administrativo **12**

6.1 Módulos 12

6.2 Princípios de coesão 13

7. Segurança e conformidade (OWASP) 13

7.1 Postura de segurança . . . 13

7.2 Gates de CI de segurança (bloqueantes) . . . 13

7.3 Headers HTTP e configuração do proxy . . . 14

7.4 Validação de entrada . . . 14

7.5 Isolamento do pipeline de mídia . . . 14

7.6 Sessões, CSRF e rate limiting . . . 15

7.7 Criptografia em trânsito e em repouso . . . 15

7.8 Auditoria e resposta a incidentes . . . 15

7.9 Posicionamento legal . . . 15

7.10 Auditoria de segurança Cloudflare . . . 16

8. Storage e compatibilidade S3 17

8.1 Interface . . . 17

8.2 Drivers e compatibilidade . . . 17

8.3 Modelo de chaves . . . 17

8.4 Presign e expiração . . . 18

8.5 Migração entre drivers . . . 18

8.6 Configuração . . . 18

8.7 Boas práticas por instância . . . 18

9. Roadmap por fases 19

Fase 0: Fundação (1 a 2 semanas) . . . 19

Fase 1: MVP (6 a 8 semanas) . . . 19

Fase 2: Comunidade e UX (4 a 6 semanas) . . . 20

Fase 3: Moderação e robustez (4 a 6 semanas) . . . 20

Fase 4: Escala e ecossistema (contínuo) . . . 21

Fase 5: Comunidade open source (contínuo) . . . 21

10. Estratégia de testes **21**

11. Riscos e mitigação **21**

12. Estrutura sugerida do monorepo **22**

13. Workflow de desenvolvimento com IA e gates de qualidade **22**

13.1 Skill ponytail: modo "senior dev lazy" . . . 22

13.2 Skill security-audit: gate de segurança por PR . . . 23

13.3 Gates de commit (pre-commit) . . . 23

13.4 Gates de CI (bloqueantes) . . . 23

13.5 Code review por IA + review humano obrigatório . . . 24

13.6 Padrão de migrations: zero-downtime e reversível . . . 24

13.7 Branch protection e conventional commits . . . 25

13.8 Smoke test pós-deploy e rollback automático . . . 25

14. Padrões e skills essenciais **25**

14.1 Skills a instalar (obrigatórias) . . . 25

14.2 Padrões de código . . . 26

14.3 Padrões de API . . . 26

14.4 Padrões de banco de dados . . . 27

14.5 Padrões de observabilidade . . . 27

14.6 Padrões de documentação . . . 27

14.7 Padrões de acessibilidade e i18n . . . 28

14.8 Padrões de frontend Next.js (App Router) otimizado . . . 28

14.9 Padrões para app mobile (pré-planejamento) . . . 30

15. Próximos passos imediatos **31**

1. Visão e princípios

MangaShelf é uma plataforma open source de hospedagem de mangás, robusta e fácil de auto-hospedar. O projeto entrega apenas o software: não hospeda, não opera e não cura conteúdo, e cada instância é de responsabilidade de quem a instala.

Princípios:

- **onolito modular**Um repositório, poucos binários, sem microsserviços.
- **ortas e adaptadores**O código de negócio depende de interfaces; o `.env` escolhe a implementação na inicialização.
- **adrões sensatos, extras opcionais**Tudo funciona com o conjunto mínimo (Postgres + disco local); Meilisearch, Redis, imgproxy e OIDC são ativados por flag.
- **utorização granular desde o dia 1**Nenhuma checagem do tipo `role == admin`; só permissões atômicas com escopo.
- **egurança e conformidade por design**O OWASP é o mínimo obrigatório (seção 7). Upload isolado, moderação e auditoria fazem parte do núcleo, não de um "depois".
- **omece simples, abstraia cedo**Postgres FTS antes de Meilisearch, disco local antes de S3, mas sempre atrás de interface.
- **ainel administrativo coeso**A administração é um módulo de primeira classe, com design e comportamento únicos (seção 6).
- **IP nativo**Acesso antecipado e conteúdo exclusivo fazem parte do modelo de acesso desde o início (seção 5).
- **oftware, não serviço**O projeto fornece o código e modelos de documentos; a operação e o conteúdo de cada instância são responsabilidade do operador. Questões judiciais relativas a conteúdo hospedado em uma instância são do operador, não do projeto — o README, o LICENSE e os modelos de termos deixam isso explícito.

2. Stack e decisões

Camada	Escolha	Observação
API	Go + `net/http` (1.22+) com `chi`	Leve e fácil de contribuir
Banco	PostgreSQL + `pgx` + `sqlc`	Queries tipadas, sem ORM
Migrations	`goose` ou `atlas`	Schema versionado
Jobs	`River` (fila em Postgres)	Redis deixa de ser requisito
Sessões	`scs` com store em Postgres	Redis opcional

Camada	Escolha	Observação
Imagens	`imgproxy` (padrão) ou modo `static` com `govips`	Veja seção 3
Storage	Local ou qualquer S3-compatível (AWS S3, R2, MinIO, Garage, SeaweedFS, B2), veja seção 8	Banco guarda só a chave
Busca	Postgres FTS (padrão) ou Meilisearch	`unaccent` + `pg_trgm`
Auth	Builtin (padrão) ou OIDC	Autorização sempre própria
Contrato	OpenAPI + `oapi-codegen` + client TS gerado	Front e back sincronizados
Front	Next.js (App Router) + TypeScript + Tailwind + TanStack Query	SSR/ISR para SEO
Proxy	Caddy	HTTPS automático
CDN	Cloudflare na frente	Banda de imagens é o gargalo
Observabilidade	`slog`, Prometheus, Grafana, Uptime Kuma	—
Backup	`pg_dump` + `restic`	Para storage externo

Por que Next.js e não React puro: páginas de série e capítulo precisam ser indexáveis. O Next entrega SSR/ISR pronto; um SPA exigiria montar isso manualmente. Não use o otimizador de imagens do Next para as páginas dos mangás; esse papel é do imgproxy ou do modo static.

3. Arquitetura e sistema de flags

3.1 Flags de configuração

```
SEARCH_DRIVER=postgres        # postgres | meilisearch
IMAGE_PIPELINE=imgproxy       # imgproxy | static
STORAGE_DRIVER=s3             # local | s3
CACHE_DRIVER=memory           # memory | redis
AUTH_DRIVER=builtin           # builtin | oidc
```

O composition root (`main.go`) lê a config, valida e instancia os adaptadores. O resto do código só conhece as interfaces.

3.2 Interfaces (portas)

- `Storage` put, get, delete, presign.
- `SearchEngine` index, delete, search, `Capabilities()`.
- `ImageURLBuilder` monta a URL final conforme o pipeline ativo.
- `Cache` memória ou Redis.
- `Authenticator` builtin ou OIDC.
- `Authorizer` sempre implementação própria.

```
// Exemplo: ImageURLBuilder — a URL nunca é persistida, só gerada em tempo de resposta.
type ImageURLBuilder interface {
    // Build retorna a URL pública ou assinada para uma página em uma variante.
```

```
    // variant ∈ {"thumb", "medium", "full"}; page contém storage_key e flags de acesso.
    Build(ctx context.Context, page Page, variant string) (string, error)
}

// Implementação imgproxy: HMAC(key/salt) sobre path+resize+expiração.
// Implementação static: URL direta do CDN usando a chave pré-renderizada no upload.
```

3.3 Regras importantes

- **O banco nunca guarda URL, só** `storage_key`**.:**O `ImageURLBuilder` gera a URL em tempo de resposta.
- **imgproxy:**URLs assinadas com HMAC (key/salt) para evitar abuso de CPU; cache na frente (Cloudflare ou Caddy), pois o imgproxy não tem cache próprio.
- **Modo static:**o worker gera as variantes (tamanhos e WebP/AVIF) no upload, e o front serve direto do storage/CDN.
- **Meilisearch:**a sincronização acontece por outbox ou jobs, nunca no request. Se o Meilisearch cair, nada quebra. Comando `cli search reindex` para quem ativar o driver com dados existentes.
- **Capabilities:**se um recurso não existe em um driver (ex.: facetas avançadas no FTS), a API expõe via `Capabilities()` e o front esconde o recurso.
- **Docker Compose profiles:**`--profile imgproxy`, `--profile meilisearch`, `--profile redis`. O serviço só sobe se for usado.
- **Validação de config no boot:**combinações inválidas falham com mensagem clara.

3.4 Módulos internos do Go

`auth`, `authz`, `catalog` (séries e capítulos), `media` (upload e processamento), `library` (follows e progresso), `moderation`, `search`, `notifications`.

Também existem `vip` (planos, assinaturas e regras de acesso) e `admin` (endpoints agregadores do painel administrativo).

3.5 Fluxo de upload

1. Usuário envia ZIP ou imagens (preferencialmente por URL pré-assinada).
2. A API valida tamanho, magic bytes e limite de páginas, e cria um job.
3. O worker extrai, remove metadados, normaliza, ordena as páginas e grava no storage com nome baseado em hash do conteúdo.
4. Se `IMAGE_PIPELINE=static`, gera também as variantes.
5. O capítulo passa de `processing` para `published`.

```
// Esqueleto do worker de mídia (internal/media/worker.go)
func (w *Worker) ProcessChapter(ctx context.Context, job river.Job) error {
    ch, err := w.repo.Chapter(ctx, job.ChapterID)
    if err != nil {
        return fmt.Errorf("load chapter: %w", err)
    }
    files, err := w.zip.SafeExtract(ctx, job.StorageKey) // protege contra zip bomb
    if err != nil {
        return w.fail(ch, err)
```

```
    }
    files = media.StripMetadata(files)                    // remove EXIF
    media.NormalizeOrder(files)                           // ordena páginas
    for _, f := range files {
        key := media.ContentKey(f)                        // sha256 → chave deduplicável
        if err := w.storage.Put(ctx, key, f); err != nil {
            return w.fail(ch, err)
        }
    }
    if w.cfg.ImagePipeline == "static" {
        if err := w.variants.Generate(ctx, files); err != nil { // WebP/AVIF + tamanhos
            return w.fail(ch, err)
        }
    }
    return w.publish(ctx, ch) // processing → published
}
```

3.6 Fluxo de leitura

O front busca a lista de páginas na API e carrega as imagens direto do imgproxy/CDN com cache `immutable`. Os bytes das imagens nunca passam pelo Go.

```
Browser ──GET /api/chapters/{id}/pages──▶ API Go
            (valida CanRead, devolve [{page, url_assinada}])
Browser ──GET https://img.example.com/...──▶ imgproxy / CDN
            (bytes da imagem; nunca passam pela API)
```

3.7 Compatibilidade com app mobile

O backend MangaShelf é **mobile-ready desde o MVP**. Embora o app mobile (React Native + Expo) chegue depois da Fase 2, todos os endpoints, contratos de autenticação e variante de imagem já suportam mobile desde a Fase 1. O custo de fazer isso desde o início é marginal; o custo de retrofit é alto (quebra de contrato, re-trabalho de auth).

A skill `mangashelf-mobile-api` (arquivo `skills-mangashelf/mangashelf-mobile-api/SKILL.md`) é o guardião: toda sessão de IA que toca API ou storage segue essa skill. Os pontos abaixo são o resumo executivo; o detalhe está na skill.

Autenticação mobile (access + refresh rotativo)

- Access token JWT (Ed25519) com `sub`, `scope`, `exp` em 15 min.
- Refresh token opaque (32 bytes `crypto/rand`), 30 dias, guardado no banco com `device_id` e `rotated_at`.
- Rotação: cada uso do refresh gera um novo e invalida o anterior. Reúso de refresh antigo = suspeita de roubo = revoga todos os tokens do usuário.
- Logout revoga o refresh token daquele device. Access token continua válido até expirar (denylist em Redis opcional para revogação imediata).
- Headers obrigatórios: `Authorization: Bearer` , `X-Device-Id`, `X-App-Version`, `X-Platform` (ios|android).

Endpoints mobile (prefixo /api/v1/mobile/)

- Endpoints otimizados para mobile são prefixados com `/mobile/` e podem ter campos extras (`device_id`, `app_version`, `platform`).

- Rate limit por device (não só por user): login 5/min/IP, refresh 30/min/user+device, páginas 60/min/user.
- ETag em todo GET de listagem; responde 304 com `If-None-Match` para economia de dados móveis.
- Compactação gzip/brotli habilitada no proxy para clients que aceitam.
- Erros seguem RFC 9457 (problem+json) com campo `code` estável para branching no app (não usar status code para isso).

Variantes de imagem para mobile

Variante	Largura	Uso
thumb	240w	Catálogo mobile
medium	480w	Card de série mobile
mobile-full	800w	Leitor mobile (prioridade)
desktop-full	1200w	Leitor web/desktop
original	original	Download (VIP)

iOS 14+ e Android 12+ suportam AVIF; versões antigas recebem WebP. imgproxy negocia via `Accept` header. Mobile pede `mobile-full` por padrão; leitor web pede `desktop-full`.

Push notifications (FCM + APNs)

- Interface `PushProvider` com drivers `fcm`, `apns`, `both`, `none` (default em dev). Config via `PUSH_DRIVER`.
- Tabela `device_push_tokens` guarda `(user_id, device_id, platform, token, last_seen)`; tokens inativos (> 30 dias) são expirados.
- Eventos que disparam push: novo capítulo de série seguida, capítulo `early` virou `public` (para não-VIP), VIP expira em 3 dias, resposta a denúncia/comentário.
- Payload máximo 4KB; `data` (não `notification`) para controle total no app; deep link `mangashelf://chapter/{id}`.
- Push tokens guardados com AES-256-GCM em repouso (V6.5.1 do ASVS).

Sync incremental (offline-first)

Endpoint `GET /api/v1/mobile/sync?since=` devolve delta desde último sync: progresso de leitura, follows adicionados/removidos, capítulos novos de séries seguidas. O app mantém SQLite local (WatermelonDB ou expo-sqlite) com mesma estrutura; conflitos de progresso resolvem por "última escrita vence", follows são union.

Versionamento e deprecação

- URL versionada: `/api/v1/...` para OpenAPI 1.x.y; `/api/v2/...` para 2.x.y.
- Bump de URL SÓ com breaking change. Versão antiga permanece por 6 meses com header `Sunset`.
- Header `X-App-Version` permite ao server decidir comportamento por versão (deprecatar gradual).
- Antes de remover campo da resposta, marcar `deprecated: true` no OpenAPI por 1 release. `oasdiff` (skill `mangashelf-openapi-diff`) detecta e avisa.

App mobile (React Native + Expo) — planejado para pós-Fase 2

- Stack: Expo (managed workflow) + React Native + TypeScript + NativeWind + TanStack Query.
- Navegação: Expo Router (file-based, similar ao Next App Router).
- Estado local: Zustand para UI state; TanStack Query para cache de servidor.
- Banco local: expo-sqlite (SQLite) para progresso offline e cache de capítulos lidos recentemente.
- Push: `expo-notifications` abstrai FCM/APNs; server fala FCM/APNs diretamente.
- Auth: `expo-secure-store` para refresh token; access token em memória.
- Imagens: `expo-image` com `source.uri` apontando para imgproxy; cache automático.
- Deep link: `expo-linking` registra `mangashelf://` scheme.
- OTA updates: EAS Update para hotfix de JS sem passar pela App Store/Play Store.
- Build: EAS Build para iOS (requer conta Apple Developer) e Android (requer conta Google Play Console).

4. Autenticação e autorização

4.1 Autenticação

- `builtin` **(padrão):**`scs` para sessões em cookie httpOnly, `argon2id`, `goth` para OAuth social, `go-webauthn` para passkeys, `pquerna/otp` para TOTP. Você escreve a cola, não a criptografia.
- `oidc` **(opcional):**aceita Keycloak, Authentik, Zitadel e outros IdPs.
- **Alternativa:**trocar o builtin por Ory Kratos (headless), se você não quiser manter recuperação de senha e MFA. A UI continua sendo sua.

A auth vive no Go, não no Next, para existir uma única fonte de verdade.

4.2 Autorização por permissões atômicas

O código **nunca checa role**:

```
if !authz.Can(ctx, user, "chapter.publish", authz.Group(groupID)) {
    return ErrForbidden
}
```

```
permissions      (key text primary key)
roles            (id, name, position int)
role_permissions (role_id, permission_key)
user_roles       (user_id, role_id)                -- escopo global
group_members    (group_id, user_id, role_id)      -- escopo por grupo
user_overrides   (user_id, permission_key, effect)  -- allow/deny pontual
```

Regras de design:

- Role é só um nome para um pacote de permissões, editável pelo dono da instância.
- A mesma permissão pode valer globalmente ou só dentro de um grupo.
- **Anti-escalação:** ninguém concede o que não tem nem edita cargo acima do seu (`position`).
- **Deny vence allow.**
- O primeiro dono é criado via `cli bootstrap-owner`, sem role mágico no código.

- Toda ação sensível vai para o `audit_log`.

```
// Ordem de avaliação do resolver (internal/authz/resolve.go)
// 1. user_overrides (deny vence allow) → 2. group_members → 3. user_roles → 4. negação
func (r *Resolver) Can(ctx context.Context, u User, perm string, scope Scope) bool {
    if eff, ok := r.overrides.Effect(u.ID, perm); ok {
        return eff == EffectAllow // override pontual tem precedência
    }
    if r.groupAllows(ctx, u, perm, scope) { return true }
    if r.roleAllows(ctx, u, perm) { return true }
    return false
}
```

5. VIP e controle de acesso ao conteúdo

O VIP é um cargo com a permissão `content.read_vip` e prazo de validade; o código nunca checa "é VIP" diretamente, e toda decisão de leitura passa por um único resolver.

5.1 Regras definidas pelo publicador

Configuração	Quem lê	Quando
Série pública, capítulo público	Todos	Ao publicar
Capítulo com acesso antecipado (`early` + `public_at`)	VIPs de imediato; demais usuários só a partir de `public_at`	Demais só a partir de `public_at`
Capítulo só VIP (`vip_only`)	Apenas VIPs	Sem abertura pública, ou com `public_at` opcional
Série só VIP (`vip_only`)	Apenas VIPs	Todos os capítulos herdam a regra

5.2 Como funciona

- **Resolver único:** `access.CanRead(user, chapter)` é usado pela API, pelo leitor, pela busca e pelos feeds. Nenhuma outra parte do código decide acesso.

```
// Resolver único de acesso (internal/vip/access.go)
func CanRead(ctx context.Context, u User, ch Chapter) bool {
    if isStaffOrEditor(ctx, u, ch.GroupID) { return true }        // equipe vê sem ser VIP
    switch ch.Access {
    case "public":
        return true
    case "early":
        if u.HasActiveVIP(ctx) { return true }                    // VIP não espera
        return ch.PublicAt != nil && !time.Now().Before(*ch.PublicAt)
    case "vip_only":
        if ch.Series.Access == "vip_only" { return u.HasActiveVIP(ctx) }
        return u.HasActiveVIP(ctx) || (ch.PublicAt != nil && !time.Now().Before(*ch.PublicAt))
    }
    return false
}
```

- **Liberação sem depender de job:** o acesso público é calculado na leitura (`now >= public_at`). Um job agendado em `public_at` só revalida o cache do Next, envia notificações e atualiza sitemap e índice de busca.

- **Expiração:** `vip_memberships.expires_at` vale na hora; um job avisa o usuário antes de vencer.
- **Concessão:** manual pelo admin, por voucher ou, opcionalmente, por provedor de pagamento (`PaymentProvider`, com `PAYMENTS_DRIVER=none` por padrão).
- **Escopo do publicador:** definir acesso e `public_at` é a permissão `chapter.set_access`, com escopo de grupo, então cada grupo controla só os próprios projetos.
- **Equipe:** quem tem permissão de editar ou moderar o conteúdo também o vê, sem ser VIP.

5.3 Proteção das imagens

- A API só devolve a lista de páginas a quem pode ler; sem ela, as chaves do storage não vazam.
- Chaves baseadas em hash de conteúdo, não adivinháveis, mais URLs assinadas de curta duração para conteúdo VIP e antecipado.
- Conteúdo restrito sai com `Cache-Control: private` ou com assinatura validada na borda, para a CDN nunca servir a imagem a quem não pode ler.
- Ao se tornar público, o capítulo passa a usar URLs públicas e cacheáveis.
- **Limite honesto:** quem pode ler consegue salvar as imagens. O objetivo é barrar o acesso casual, não é DRM. Mitigações: rate limit por conta, limite de sessões simultâneas e marca d'água opcional por flag.

5.4 Modelo de dados

```
series           add: access text                -- 'public' | 'vip_only'
chapters         add: access text                -- 'public' | 'early' | 'vip_only'
chapters         add: public_at timestamptz null
vip_plans        (id, name, duration, perks jsonb)
vip_memberships  (user_id, plan_id, starts_at, expires_at, source)  -- admin | voucher | payment
vip_vouchers     (code_hash, plan_id, max_uses, expires_at)

-- VIP ativo na hora da leitura (sem depender de job):
-- expires_at > now() é avaliado no resolver CanRead.
```

6. Painel administrativo

O painel é um módulo de primeira classe do MangaShelf: um único app em `/admin`, com design, navegação e padrões iguais em todas as telas.

6.1 Módulos

Módulo	O que resolve	Permissão-base
Visão geral	Métricas de leitura, uploads, filas e alertas	`admin.view`
Catálogo	Séries, capítulos, tags, autores, grupos, acesso e agendamento	`series.edit`, `chapter.publish`
Mídia	Fila de processamento, falhas, reprocessamento, uso de storage	`media.manage`
Usuários	Busca, perfil, sessões, banimento, concessão de VIP	`user.view`, `user.ban`

Módulo	O que resolve	Permissão-base
Cargos e permissões	Editor de cargos, matriz de permissões, overrides	`role.manage`
VIP	Planos, vouchers, assinaturas e expirações	`vip.manage`
Moderação	Denúncias, fila e remoção de conteúdo	`report.handle`
Auditoria	Log filtrável por usuário, ação e período	`audit.view`
Instância	Configurações em runtime, estado dos drivers, reindex de busca	`instance.configure`
Sistema	Saúde, jobs, versões e backups	`system.view`

6.2 Princípios de coesão

- **Um app só:**rota `/admin` no mesmo Next.js, com o mesmo design system e o mesmo client gerado do OpenAPI; sem painel paralelo.
- **O servidor decide:**a UI esconde o que o usuário não pode fazer, mas toda ação é validada pelo `Authorizer` nos endpoints `/admin/*`.
- **Padrão único de telas:**tabela com busca, filtros, paginação por cursor, ordenação e ações em lote; estado na URL; atalhos de teclado.
- **Navegação por permissão:**cada pessoa vê só os módulos que pode usar.
- **Auditoria total:**toda mutação registra autor, alvo e valores antes e depois.
- **Drivers do `.env` só leitura:**o painel mostra o driver ativo e a saúde de cada um; o que for editável em runtime fica no banco.
- **Acessível e responsiva:**usável em tablet e celular para moderação rápida.

7. Segurança e conformidade (OWASP)

O OWASP ASVS **nível 2** é o mínimo obrigatório e o build segue essas regras como gate de CI bloqueante. Segurança não é uma fase: upload isolado, moderação e auditoria são núcleo, e os requisitos abaixo se aplicam desde o MVP.

7.1 Postura de segurança

- **Defense in depth:**proxy (Caddy) → API (validação + autorização) → banco (RLS/constraints) → storage (chaves não adivinháveis). Nenhuma camada confia na anterior.
- **Fail-safe defaults:**qualquer erro de autorização nega; qualquer config ambígua falha no boot.
- **Least privilege:**o worker não tem credencial de admin do banco; a API não tem credencial de escrita no storage de variantes estáticas; o painel só expõe o que a permissão permite.
- **Superfície mínima:**sem endpoint debug, sem admin "escondido" em porta separada — o `/admin/*` vive atrás do mesmo Authorizer com permissões atômicas.

7.2 Gates de CI de segurança (bloqueantes)

Gate	Ferramenta	Critério de bloqueio
SAST	`gosec` + `semgrep`	Qualquer achado crítico/alto bloqueia o merge
SCA Go	`govulncheck`	Vulnerabilidade afetando symbol em uso bloqueia
SCA TS	`npm audit` / `pnpm audit --prod`	CVE alto/crítico em dependência de produção bloqueia
Container scan	`Trivy` nas imagens docker	CVE crítico com fix disponível bloqueia
Secret scanning	`gitleaks`	Qualquer segredo detectado bloqueia
Dependências	Dependabot/Renovate	PR automático; janela de patch de 7 dias para críticos

7.3 Headers HTTP e configuração do proxy

O Caddy aplica, no mínimo:

```
header {
    Strict-Transport-Security "max-age=63072000; includeSubDomains; preload"
    Content-Security-Policy   "default-src 'self';
                               img-src 'self' https://img.{dominio} data:;
                               object-src 'none';
                               frame-ancestors 'none';
                               base-uri 'self'"
    X-Content-Type-Options    "nosniff"
    X-Frame-Options           "DENY"
    Referrer-Policy           "strict-origin-when-cross-origin"
    Permissions-Policy        "camera=(), microphone=(), geolocation=()"
    Cross-Origin-Opener-Policy "same-origin"
    -Server
}
```

- CSP sem `unsafe-inline` em produção; nonces/hashes gerados pelo Next.
- Cookies: `Secure`, `HttpOnly`, `SameSite=Lax` (sessão); `SameSite=Strict` para ações do `/admin`.
- Domínio de mídia separado (`img.exemplo.com`): sem cookies, sem JS — limita impacto de XSS.

7.4 Validação de entrada

- Contrato OpenAPI como única fonte: `oapi-codegen` valida corpo, query e path contra schemas estritos (sem campos extras).
- Uploads: tamanho máximo por arquivo e por capítulo, whitelist de magic bytes (ZIP + imagens), contagem máxima de páginas, nomes de arquivo sanitizados.
- Textos: normalização UTF-8, limites de tamanho, sanitização na renderização (sem HTML cru em título/sinopse; Markdown com whitelist de tags).
- IDs sempre validados por escopo: 404 em vez de 403 para recursos de outros grupos, evitando enumeração.

7.5 Isolamento do pipeline de mídia

- Worker roda em container próprio, sem credencial de admin, com timeout por job e limite de memória/CPU.
- Extração de ZIP protegida contra zip bomb (limite de descompressão, ex.: 200×, e limite de arquivos).
- Remoção de metadados (EXIF/GPS) em todas as páginas no processamento.
- Re-encode de imagem no modo static elimina payloads maliciosos escondidos.
- Storage acessível apenas por chaves geradas no worker; sem listagem pública de bucket.

7.6 Sessões, CSRF e rate limiting

- `scs` com rotação de session ID no login e no upgrade de privilégio.
- CSRF: double-submit token em todas as mutations do browser; endpoints `/admin/*` exigem header customizado.
- Rate limit por IP **e** por conta (login, cadastro, busca, API pública), com backoff progressivo; limites reforçados em `cli` e webhooks.
- Verificação de e-mail obrigatória antes de ações sensíveis; recuperação de senha com token de uso único e curta duração.
- MFA (TOTP/passkeys) disponível no builtin e obrigatório para contas com `role.manage` ou `instance.configure`.

7.7 Criptografia em trânsito e em repouso

- TLS 1.2+ (preferencial 1.3) obrigatório via Caddy/Let's Encrypt; HSTS com preload.
- Senhas: `argon2id` com parâmetros revisados por release (memória/tempo publicados em ADR).
- Tokens de sessão e URLs assinadas: aleatórios criptográficos (`crypto/rand`), TTL curto (URLs VIP/early: 5–15 min).
- Em repouso: Postgres e storage em disco com criptografia a critério do operador (LUKS/dm-crypt ou SSE no provedor S3); o projeto documenta o caminho recomendado.
- Segredos nunca em código nem em log: `.env` fora do repositório, `gitleaks` no CI, rotação documentada.

7.8 Auditoria e resposta a incidentes

- `audit_log` append-only: autor, ação, alvo, valores antes/depois, IP e user-agent. Retenção mínima de 12 meses, exportável.
- Runbooks versionados em `docs/`: recuperação de conta, vazamento de chave imgproxy, incidente de conteúdo, restore de backup.
- `SECURITY.md` com política de divulgação responsável e SLA de resposta.
- Alertas (Prometheus + Uptime Kuma): pico de 4xx/5xx, fila de jobs travada, falha de storage, volume anômalo de downloads de um único usuário.

7.9 Posicionamento legal

- **O projeto distribui software; não opera serviço.** Nada no código hospeda conteúdo dos mantenedores.
- `LICENSE` (AGPL-3.0) + aviso de ausência de garantia + `README` com aviso explícito de responsabilidade do operador.
- Modelos de termos de uso, política de privacidade e política de conteúdo (seção 3 da Fase 3) são fornecidos como templates editáveis — cada operador responde pelo próprio uso.
- Ferramentas de moderação, takedown e auditoria fazem parte do produto exatamente para dar ao operador meios de cumprir suas obrigações.

7.10 Auditoria de segurança Cloudflare

O MangaShelf usa Cloudflare na frente de toda instância de produção (seção 2). A auditoria Cloudflare é um gate obrigatório antes de cada release e usa o skill `security-audit` (instalado por `clawhub install security-audit`) como base, complementado pelo checklist específico abaixo. O operador executa; o projeto documenta.

Camada Cloudflare	Configuração esperada	Frequência de revisão
SSL/TLS	Modo Full (strict); TLS 1.3 habilitado; HSTS preload; minimum TLS version 1.2	Quarterly
WAF	Managed Ruleset (Cloudflare) + OWASP Core Ruleset ativados; custom rules para bloquear path traversal e SQLi em `/admin/*`	Quarterly
Bot Fight Mode	Ativado para domínio da aplicação; Super Bot Fight Mode para domínio de mídia (`img.*`)	Quarterly
Rate Limiting	Regras para login (10/min/IP), cadastro (5/min/IP), busca (60/min/conta) e API pública (token bucket por escopo)	Quarterly
DNSSEC	Habilitado no domínio apex; CAA records restringem CA a Let's Encrypt	On change
Cache Rules	Assets estáticos com `Cache-Control: immutable, max-age=31536000`; HTML do Next com `s-maxage=60, stale-while-revalidate=60`	Quarterly
Transform Rules	Headers de segurança (CSP, X-Frame-Options, Referrer-Policy) injetados na borda como defense in depth com o Caddy	Quarterly
Cloudflare Access	Protege `/admin/*` com zero-trust (OIDC + lista de e-mails autorizados) na frente do Authorizer	On change
Workers	Worker na borda valida HMAC das URLs assinadas (seção 5.3) antes de servir conteúdo VIP/early — a CDN nunca serve a quem não pode ler	Quarterly
Logpush	Logs de WAF, access e firewall exportados para R2/S3 (raw JSON); retenção de 90 dias	On change
Page Rules / Redirect Rules	Auditar regras legadas; substituir por Cache Rules e Redirect Rules (mais expressivas)	Quarterly
Radar / Threat intel	Monitorar ataques em campanha via Cloudflare Radar; ajustar WAF conforme tendências	Weekly

Como executar: o skill `security-audit` (seção 13) roda em CI contra o `.env.example` e os arquivos de deploy, validando segredos, configurações fracas e permissões. Para a camada Cloudflare em si, o operador executa manualmente o checklist acima a cada release; o projeto fornece o template em `docs/runbooks/cloudflare-audit.md`.

8. Storage e compatibilidade S3

O armazenamento é abstraído pela interface `Storage`, e qualquer backend S3-compatível deve funcionar sem código novo: AWS S3, Cloudflare R2, MinIO, Garage, SeaweedFS e Backblaze B2 são os alvos de teste na matriz de CI "completa".

8.1 Interface

```
type Storage interface {
    Put(ctx context.Context, key string, r io.Reader, size int64) error
    Get(ctx context.Context, key string) (io.ReadSeekCloser, error)
    Delete(ctx context.Context, key string) error
    // Presign gera URL de upload (PUT) ou leitura (GET) com expiração curta.
    Presign(ctx context.Context, key string, op Op, ttl time.Duration) (string, error)
    List(ctx context.Context, prefix string) ([]ObjectInfo, error)
}
```

8.2 Drivers e compatibilidade

Driver	Alvos	Observações
`local`	Disco local/NFS	Path canônico sob `DATA_DIR`; chmod restritivo
`s3`	AWS S3, R2, MinIO, Garage, SeaweedFS, B2	Via `aws-sdk-go-v2` com `endpoint` customizável

Compatibilidade garantida por:

- `S3_FORCE_PATH_STYLE=true` por padrão (necessário para MinIO, Garage, SeaweedFS; opcional para AWS).
- `S3_REGION` configurável (R2 usa `auto`; B2 usa região própria via endpoint S3).
- Sem uso de recursos exclusivos da AWS (sem RRS, sem Object Lock, sem S3 Select).
- Testes de contrato da seção 10 rodam a mesma suíte contra MinIO em Docker e local — qualquer divergência de comportamento quebra o CI.
- Verificação de capabilities no boot (multipart suportado? presign suportado?) com mensagem clara.

8.3 Modelo de chaves

- **O banco nunca guarda URL, só** `storage_key`, composta por hash de conteúdo: `pages/{series_id}/{chapter_id}/{sha256}.webp`.
- Hash de conteúdo ⇒ deduplicação automática (mesma página não é gravada duas vezes) e chave não adivinhável (sem ID sequencial vazável).

- A URL final (domínio, assinatura, variante) é calculada pelo `ImageURLBuilder` em tempo de resposta — trocar de driver ou de CDN não exige migrar dados.

8.4 Presign e expiração

Conteúdo	URL	TTL
Público	CDN/imgproxy com `Cache-Control: public, max-age=31536000, immutable`	—
VIP / early	Presign `GET` com assinatura	5–15 min
Upload	Presign `PUT` por arquivo	15 min

- Uploads grandes usam multipart presigned (partes de 5–50 MB), essencial em R2/MinIO.
- Assinatura validada na borda quando possível (Cloudflare Worker ou Caddy `forward_auth`) para conteúdo restrito, impedindo que a CDN sirva imagem a quem não pode ler.

8.5 Migração entre drivers

```
# Copia tudo do driver ativo para um destino, em background, com retomada.
cli storage migrate --from local --to s3 --parallel 4 --resume
# 1. Lista chaves no driver origem (prefixo por série para particionar).
# 2. Compara existence/ETag no destino; copia só o que falta (stream, sem carregar em RAM).
# 3. Reescreve apenas variáveis de ambiente; storage_key no banco não muda.
```

Migração é indolor por design: como o banco só guarda chave, trocar `STORAGE_DRIVER` após a cópia concluída é suficiente.

8.6 Configuração

```
STORAGE_DRIVER=s3
S3_ENDPOINT=https://<accountid>.r2.cloudflarestorage.com   # ou http://minio:9000
S3_REGION=auto          # auto (R2) | us-east-1 (AWS) | us-east-1 (MinIO padrão)
S3_BUCKET=mangashelf
S3_ACCESS_KEY=...       # sem valor ⇒ usa IAM role / instance profile quando disponível
S3_SECRET_KEY=...
S3_FORCE_PATH_STYLE=true
S3_PUBLIC_BASE_URL=https://cdn.exemplo.com   # opcional: base de CDN pública
```

8.7 Boas práticas por instância

- **Bucket privado por padrão:**todo acesso público passa por CDN/imgproxy com URL assinada; nunca bucket público de leitura.
- **Lifecycle:**expirar uploads multipart incompletos (7 dias) e `processing` órfão (14 dias).
- **Versionamento opcional:**recomendado em produção para recuperação rápida de deleção acidental.
- **Particionamento por prefixo:**(`series_id/chapter_id`) mantém listagem eficiente em B2/Garage, que penalizam list amplas.
- **Métricas de uso:**(bytes por série/capítulo) expostas no painel de Mídia para o operador dimensionar custo de egress.

9. Roadmap por fases

Fase 0: Fundação (1 a 2 semanas)

Entregas

- Licença AGPL-3.0 (protege contra forks fechados oferecidos como serviço), com `LICENSE`, aviso de ausência de garantia e política de contribuição (DCO).
- Monorepo: `apps/api`, `apps/worker`, `apps/web`, `deploy/`, `docs/`.
- `docker-compose` de desenvolvimento com profiles.
- CI: lint, testes, build e verificações de segurança bloqueantes (seção 7), com matriz "mínimo" e "completo".
- `README`, `CONTRIBUTING`, `CODE_OF_CONDUCT`, `SECURITY.md` e ADRs curtos com as decisões deste documento.
- Definição das interfaces (seção 3.2) e loader de config com validação.
- Modelagem inicial do banco e esqueleto do OpenAPI.
- Geração de client TS a partir do OpenAPI integrada ao build.

Pronto quando: `docker compose up` sobe API, worker, web e Postgres vazios, e a CI passa.

Fase 1: MVP (6 a 8 semanas)

Auth e autorização

- Cadastro, login, logout, verificação de e-mail e recuperação de senha (`builtin`).
- Modelo completo de permissões, roles, escopo por grupo e overrides.
- `cli bootstrap-owner`, audit log básico.

Catálogo

- CRUD de séries, títulos alternativos, autores, tags e grupos.
- CRUD de capítulos com estados (`draft`, `processing`, `published`, `removed`).

Mídia

- Upload por URL pré-assinada, validação, extração de ZIP e job no worker.
- Pipeline de imagem com os dois modos (`imgproxy` e `static`) e `ImageURLBuilder`.
- Storage `local` e `s3`.

Leitor e site

- Leitor com modo paginado e scroll vertical, atalhos de teclado, layout mobile e pré-carregamento das próximas páginas.
- Home, catálogo, página da série, página do capítulo, SEO (metadados, Open Graph, sitemap).
- Busca com Postgres FTS (`unaccent` + `pg_trgm`).

Operação

- Deploy em VPS com Caddy e HTTPS, backups automatizados e restauração testada.

- Logs estruturados e healthchecks.

VIP e acesso

- Campos de acesso e agendamento em séries e capítulos, resolver único `CanRead` e proteção das imagens (seção 5).
- VIP concedido manualmente pelo admin e por vouchers, com expiração.
- Job de liberação em `public_at` para revalidar cache, notificar e atualizar índices.

Painel admin v1

- Shell em `/admin` com layout, navegação por permissão e padrão de tabelas (seção 6).
- Módulos: visão geral, catálogo, fila de mídia, usuários e auditoria básica.

Segurança

- Gates de segurança da CI ativos e headers de segurança no proxy (seção 7).

Pronto quando: um uploader autorizado publica um capítulo, um visitante o lê em qualquer dispositivo, um capítulo com acesso antecipado abre para o público só na data marcada, e o ambiente restaura de backup.

Fase 2: Comunidade e UX (4 a 6 semanas)

- Biblioteca: follows, listas, histórico e progresso de leitura sincronizado.
- Filtros por tags, status, ano e demografia.
- Driver **Meilisearch**, `cli search reindex` e outbox de sincronização.
- Driver **OIDC**, OAuth social, passkeys e TOTP.
- Comentários e notificações de novos capítulos (in-app e e-mail).
- i18n, tema claro/escuro e PWA.
- Driver de cache Redis (opcional).
- VIP com planos e níveis (antecedência diferente por plano), aviso por e-mail antes de expirar.
- Interface `PaymentProvider` com `PAYMENTS_DRIVER=none` por padrão e um primeiro adaptador opcional.
- Painel admin v2: VIP, moderação, configurações da instância e saúde do sistema.

Pronto quando: trocar `SEARCH_DRIVER` e `AUTH_DRIVER` funciona sem mudar código, com testes cobrindo ambos os caminhos.

Fase 3: Moderação e robustez (4 a 6 semanas)

- UI de gestão de cargos e permissões com regras anti-escalação.
- Audit log navegável e filtrável.
- Sistema de denúncias, fila de moderação e banimentos (com expiração e motivo).
- Ferramentas de remoção de conteúdo para o operador: formulário público opcional, remoção rápida e registro. São recursos da instância, não obrigações do projeto.
- Rate limiting, proteção contra scraping e abuso, captcha opcional.

- Modelos de termos de uso, privacidade e política de conteúdo, editáveis por instância (o operador responde por eles).
- Testes de carga, métricas (Prometheus), dashboards e alertas.
- Revisão completa contra o OWASP ASVS nível 2 e fechamento dos itens abertos (seção 7).

Pronto quando: uma denúncia ou takedown percorre o fluxo completo e fica auditado.

Fase 4: Escala e ecossistema (contínuo)

- CDN/R2 em produção e cache em camadas.
- API pública documentada com tokens e escopos.
- Webhooks, RSS/Atom e sitemap dinâmico.
- Painel admin: relatórios avançados, ações em lote e dashboards configuráveis.
- Importação e exportação de dados da instância.
- Otimização de queries e partições para tabelas grandes (páginas, comentários, progresso).

Fase 5: Comunidade open source (contínuo)

- Documentação de self-host (VPS mínima, upgrade, backup e restore, troubleshooting).
- Issues `good first issue`, templates de PR e roadmap público.
- Releases versionadas (SemVer), changelog e imagens Docker publicadas.
- Sistema de temas e possivelmente plugins.
- Avaliação futura de federação (ActivityPub).

10. Estratégia de testes

- **Unitários:**para regras de negócio e para o `Authorizer` (matriz de permissões, escopo, deny e anti-escalação).
- **Integração:**com Postgres real via testcontainers.
- **Contrato:**testes que rodam a mesma suíte contra cada implementação de `SearchEngine`, `Storage` e `ImageURLBuilder` (inclui MinIO vs. local na prática).
- **E2E:**com Playwright nos fluxos críticos: cadastro, upload, leitura, moderação.
- **CI com duas matrizes:**"mínimo" (postgres, static, local, builtin) e "completo" (meilisearch, imgproxy, s3, redis). Evite testar todas as combinações; cada flag multiplica o custo.

11. Riscos e mitigação

Risco	Mitigação
Uso indevido da plataforma por operadores de instâncias	O projeto distribui só o software, com aviso de responsabilidade no README, no LICENSE e nos termos modelo; ferramentas de moderação à disposição do operador
Banda de imagens estoura a VPS	CDN na frente, cache `immutable`, URLs com hash de conteúdo

Risco	Mitigação
Upload malicioso	Validação de magic bytes, limites, worker isolado, domínio de mídia separado, remoção de metadados
CPU alta no imgproxy	URLs assinadas, cache, conjunto fixo de tamanhos permitidos
Explosão de flags e combinações	Duas matrizes de CI, defaults fortes, validação no boot
Escalação de privilégio	Anti-escalação por `position`, deny vence allow, auditoria
Perda de dados	Backups automáticos, restore testado periodicamente
Scraping agressivo	Rate limit, Cloudflare, limites por IP e por sessão

12. Estrutura sugerida do monorepo

```
/
├── apps/
│   ├── api/        # cmd/api, internal/{auth,authz,catalog,media,library,moderation,search}
│   ├── worker/     # cmd/worker (mesmo módulo Go, outro entrypoint)
│   └── web/        # Next.js
├── packages/
│   └── api-client/ # client TS gerado do OpenAPI
├── deploy/
│   ├── docker-compose.yml
│   ├── Caddyfile
│   └── .env.example
├── api/openapi.yaml
├── migrations/
└── docs/           # ADRs, self-host, contribuição
```

13. Workflow de desenvolvimento com IA e gates de qualidade

Esta seção define como a IA (usada via Z.ai, Cursor, Copilot ou outra) trabalha no repositório sem quebrar código existente. O princípio é: nenhum PR gerado por IA entra em `main` sem passar por todos os gates below. Skills instaladas localmente garantem que a IA segue o padrão do projeto, e gates de CI bloqueiam automaticamente qualquer regressão.

13.1 Skill ponytail: modo "senior dev lazy"

Instalar o skill `ponytail` (`clawhub install @dietrichgebert/ponytail`) e ativar em toda sessão de IA que escreve código. O ponytail obriga a IA a:

- Ler o escopo completo antes de escrever — listar todos os callers, testes e fixtures que a mudança toca.
- Fazer a menor mudança que resolve o problema inteiro, sem feature creep, abstrações ou "para depois".
- Reusar helpers, componentes e padrões do próprio códigobase antes de inventar novos.
- Nunca cortar validação em trust boundary, tratamento de erro que previne perda de dado, segurança ou acessibilidade.
- Toda lógica nova não-trivial (branch, loop, parser, dinheiro, segurança) vem com pelo menos um teste.

- Bug fix: grep em todos os callers da função tocada antes de editar; corrigir causa-raiz uma vez no código compartilhado.
- Atalhos com limite conhecido ganham comentário `shortcut: , .`

Níveis: `lite` (faz o pedido, sugere menor opção), `full` (padrão, segue as regras acima), `ultra` (questiona o pedido antes de construir). Para MangaShelf, default `full`; usar `ultra` em decisões de arquitetura. O ponytail fica ativo até o usuário dizer "stop ponytail" ou "normal mode".

13.2 Skill security-audit: gate de segurança por PR

Instalar o skill `security-audit` (`clawhub install security-audit`) e rodar em todo PR antes do merge. O skill escaneia segredos expostos, portas abertas, configurações fracas, permissões de arquivo e Docker. No MangaShelf, o modo `--full --json` roda em CI e produz um relatório que bloqueia o merge se houver qualquer item CRITICAL ou HIGH.

```
# No CI, antes do build:
node skills/security-audit/scripts/audit.cjs --full --json > audit-report.json
# Falha o job se houver CRITICAL ou HIGH:
jq '.findings[] | select(.level == "CRITICAL" or .level == "HIGH")' audit-report.json

# Auto-fix opcional para problemas comuns (não-bloqueante):
node skills/security-audit/scripts/audit.cjs --fix
```

O skill cobre: credenciais em `.env`/código/histórico, portas expostas, rate limiting ausente, CORS aberto, defaults de Docker inseguros. Combinado com os gates da seção 7.2 (gosec, govulncheck, Trivy, gitleaks), forma a camada de segurança obrigatória por PR.

13.3 Gates de commit (pre-commit)

Todo commit passa por hooks locais antes de chegar ao git. Husky + lint-staged no front, gitleaks + gofmt no back:

Escopo	Hooks	Comportamento
Front (apps/web)	Prettier, ESLint, `tsc --noEmit`, `next lint`	Bloqueia commit com erro
Back (apps/api, apps/worker)	`gofmt -l`, `golangci-lint run`, `go vet ./...`	Bloqueia commit com erro
SQL (migrations/)	`sqlfluff lint` ou `squawk` (Postgres linter)	Bloqueia migration inválida
OpenAPI	`redocly lint api/openapi.yaml` + checagem de breaking diff via `oasdiff`	Bloqueia quebra de contrato
Secretos	`gitleaks protect --staged`	Bloqueia commit com chave/token
Mensagens	`commitlint` (conventional commits)	Bloqueia mensagem fora do padrão

13.4 Gates de CI (bloqueantes)

Gate	O que roda	Critério
Build	Build de API, worker e web em matriz `mínimo` e `completo`	Falha build = bloqueia
Unit + Integration	Unitários por módulo; integração com Postgres real via testcontainers	Cobertura < 70% do diff = bloqueia
Contrato	Testes de contrato de `Storage`, `SearchEngine`, `ImageURLBuilder` contra cada driver	Divergência = bloqueia
E2E (smoke)	Playwright em staging após deploy: cadastro, login, upload, leitura, moderação	Falha = rollback automático
Segurança	gosec, govulncheck, Trivy, gitleaks, npm audit, security-audit skill (seção 13.2)	CRITICAL/HIGH = bloqueia
Bundle size	`@next/bundle-analyzer`; budget de 250 KB JS por rota	Estouro = bloqueia
Acessibilidade	`axe-core` rodando nos E2E	Violação critical = bloqueia
Visual regression	Playwright snapshots (ou Chromatic) nas páginas do leitor e do admin	Diff não aprovado = bloqueia

13.5 Code review por IA + review humano obrigatório

Todo PR recebe review automática de IA (usando o skill `ponytail` em modo `ultra` ou o subagente `ai-code-reviewer` do ClawHub) antes do review humano. A IA valida:

- Aderência ao padrão de código (ESLint, golangci-lint, sqlfluff) — sem `nits` manuais.
- Cobertura de teste da mudança: todo branch novo tem teste? Toda função pública tem teste?
- Sem regressão óbvia: callers atualizados, migrations reversíveis, contratos preservados.
- Sem segredo em string ou comentário; sem `TODO` sem issue; sem `console.log` em produção.
- SQL sem N+1; queries novas com EXPLAIN; índices necessários criados.
- Aderência aos ADRs do projeto (em `docs/adr/`).

A review por IA é **consultiva** — não bloqueia sozinha. O merge exige 1 aprovação humana + todos os gates de CI verdes. Para PRs que tocam `authz`, `vip`, `media` ou migrations, exige-se 2 aprovações humanas (um mantenedor + um reviewer qualquer). Branch `main` protegida: sem push direto, sem force-push, sem merge sem PR.

13.6 Padrão de migrations: zero-downtime e reversível

Toda migration segue o padrão expand-then-contract e deve ter `up` e `down` testados em DB real antes do merge. A IA nunca gera migration que:

- Renomeia coluna/tabela em um step (quebra código antigo). Use 3 steps: add nova → backfill → drop antiga.
- Adiciona coluna `NOT NULL` sem default em um step (quebra insert antigo).
- Remove coluna ainda lida pelo código em produção. Primeiro deploy sem usar a coluna; segundo deploy remove.
- Cria índice grande em tabela viva sem `CREATE INDEX CONCURRENTLY`.

Teste obrigatório no CI: `migrate up` em DB vazia → roda testes → `migrate down` → verifica DB voltou ao estado inicial. Falha o down = bloqueia merge.

13.7 Branch protection e conventional commits

- `main` protegida: sem push direto, PR obrigatório, status checks obrigatórios, sem force-push.
- Conventional commits (`feat:`, `fix:`, `chore:`, `docs:`, `refactor:`, `perf:`, `test:`, `ci:`, `build:`) via commitlint.
- Prefixo `!` para breaking change (`feat!: ...`); rodar `oasdiff` em `api/openapi.yaml` quando há `!` ou `feat:`.
- Semantic-release opcional: gera changelog e tag a partir dos commits. Não usar para pre-1.0.
- Branches: `/` (ex.: `feat/vip-resolver`, `fix/upload-zip-bomb`).

13.8 Smoke test pós-deploy e rollback automático

Após todo deploy em produção, smoke test automático roda 5 fluxos críticos via Playwright contra a URL pública:

1. Healthcheck `/healthz` e `/readyz` retornam 200.
2. Home carrega e o catálogo tem pelo menos 1 série.
3. Login de conta de teste funciona.
4. Upload de capítulo de teste (ZIP de 3 páginas) completa em < 60s.
5. Leitura do capítulo recém-publicado funciona (páginas carregam via imgproxy).

Falha em qualquer passo aciona rollback automático (deploy da imagem anterior) e alerta no Prometheus + Uptime Kuma. O deploy nunca é "merge e reza".

14. Padrões e skills essenciais

Lista do que deve estar pronto antes da Fase 1 começar. Skills do ClawHub são instaladas localmente; padrões são documentados em ADRs e configurados nos arquivos de lint.

14.1 Skills a instalar (obrigatórias)

Skills externas (instalar via ClawHub ou manualmente; veja `skills-mangashelf/README.md` para instalação em qualquer ambiente):

Skill externa	Uso no projeto	Onde
`@dietrichgebert/ponytail`	Modo "senior dev lazy" para toda sessão de IA que escreve código	Seção 13.1
`security-audit`	Scan de segredos, portas, configs fracas, Docker — roda em CI por PR	Seção 13.2

Skills internas do projeto (em `skills-mangashelf/`, copiadas para `.cursor/skills/` ou equivalente no seu ambiente):

Skill interna	O que faz	Onde
`mangashelf-frontend-optimized`	Guardião de frontend Next.js — bloqueia useEffect para fetch, "use client" desnecessário, img sem next/image, etc.	Seção 14.9
`mangashelf-mobile-api`	Padrões de API mobile-ready — auth com refresh rotativo, push, ETag, sync incremental	Seção 3.7
`mangashelf-migration-validator`	Valida up+down de migrations em DB real; bloqueia migration irreversível	Seção 13.6
`mangashelf-adr-check`	Bloqueia PR com mudança arquitetural sem ADR	Seção 14.7
`mangashelf-openapi-diff`	Bloqueia breaking change no OpenAPI sem bump major	Seção 14.4
`mangashelf-asvs-checklist`	Checklist OWASP ASVS L2 por release; bloqueia se V1-V9 falharem	Seção 7
`mangashelf-cloudflare-audit`	Roda checklist da seção 7.10 via API Cloudflare por release	Seção 7.10
`mangashelf-perf-budget`	Lighthouse nas 5 rotas críticas; bloqueia regressão > 10% em LCP/CLS/TBT	Seção 14.9

Como instalar em qualquer ambiente (Cursor, Claude Code, Cline, Copilot Chat, Continue, Aider, etc.): veja `skills-mangashelf/README.md`. As skills são markdown puro, sem sintaxe especial — cole no system prompt ou no diretório de skills do seu editor.

14.2 Padrões de código

Camada	Padrão	Onde configurado
Go	`golangci-lint` com `gocyclo=15`, `misspell`, `gosec`, `revive`, `unused`; `gofmt` obrigatório	`.golangci.yml` no repo
TypeScript	ESLint `next/core-web-vitals` + `@typescript-eslint/recommended`; `strict: true` no `tsconfig.json`	`.eslintrc.json`, `tsconfig.json`
Prettier	Config único para front e markdown; `printWidth: 100`, `semi: true`, `singleQuote: true`	`.prettierrc.json`
SQL	`sqlfluff` com dialector `postgres`; `snake_case`, tabelas no plural, FK como `_id`	`.sqlfluff`
Commits	Conventional commits via `commitlint` com regra `type(scope): subject`	`commitlint.config.js`
Nomenclatura	Go: `mixedCaps` para exportado, `lowerCamel` para interno; TS: `camelCase` para vars, `PascalCase` para tipos/componentes	ADRs

14.3 Padrões de API

- OpenAPI estrito: `oapi-codegen` gera server e client TS; campos desconhecidos são rejeitados (não ignorados).

- Versionamento na URL: `/v1/...`. Bump de versão só com breaking change, e com janela de depreciação de 6 meses.
- Paginação cursor-based em toda listagem; sem `OFFSET` em tabelas grandes.
- Erros seguem RFC 9457 (problem+json): `type`, `title`, `status`, `detail`, `instance`. Códigos de erro estáveis em `string` (ex.: `chapter.not_found`).
- Naming: `kebab-case` nas URLs, `snake_case` no corpo JSON, `camelCase` no cliente TS (gerado).
- Rate limit por IP e por conta exposto no header `X-RateLimit-Remaining`.
- Idempotência: endpoints `POST` que criam recurso aceitam header `Idempotency-Key`.

14.4 Padrões de banco de dados

- Migrations reversíveis: todo `up` tem `down` que reverte o estado. CI roda ambos.
- Expand-then-contract para mudanças de schema (seção 13.6).
- Naming: `snake_case`, tabelas no plural (`series`, `chapters`, `users`), FK como `_id` (`series_id`).
- Índices: prefixo `idx__`; únicos com `uq__`.
- Timestamps: `created_at` e `updated_at` em toda tabela; default `now()`. Nunca usar timestamp do app.
- Soft delete apenas onde faz sentido (moderação, banimento); nunca em dados operacionais.
- Seed data em `migrations/seed/` separado, nunca em migration de schema.
- Tabelas com crescimento esperado > 1M linhas têm estratégia de particionamento documentada em ADR.

14.5 Padrões de observabilidade

- Logs: `slog` (Go) em JSON; campos obrigatórios: `request_id`, `user_id` (se autenticado), `route`, `latency_ms`, `status`. Sem PII em log.
- Métricas: Prometheus em `/metrics`; prefixo `mangashelf_`; labels por `route`, `status`, `driver`.
- Traces: OpenTelemetry opcional na Fase 2; sample rate 10% em produção.
- Healthchecks: `/healthz` (liveness) e `/readyz` (readiness); ambos sem auth.
- Runbooks: um por alerta em `docs/runbooks/.md`; inclui sintoma, diagnóstico e mitigação.
- Sentry (ou similar) para error tracking em produção; sample rate 100% para 5xx, 10% para 4xx.

14.6 Padrões de documentação

- ADRs em `docs/adr/NNNN-titulo.md` (formato Michael Nygard); um ADR por decisão arquitetural.
- README por módulo Go em `apps/api/internal//README.md` explicando responsabilidade e fronteiras.
- OpenAPI como fonte única da API; Swagger UI servido em `/docs/api` em dev.
- Runbooks em `docs/runbooks/`; cobrem: incidentes previsíveis (seção 7.8), deploy, rollback, restore de backup, reindex de busca.
- CHANGELOG.md atualizado a cada release; gerado dos conventional commits na Fase 2.

- Guia de self-host em `docs/self-host.md` com VPS mínima, variáveis de env, troubleshooting comum.

14.7 Padrões de acessibilidade e i18n

- WCAG 2.1 AA como mínimo; testes `axe-core` em CI (gate bloqueante para violações critical).
- Navegação completa por teclado no leitor e no admin; focus visível; skip link no topo.
- Imagens com `alt` descritivo; ícones com `aria-label`.
- Contraste de texto ≥ 4.5:1 (normal) e ≥ 3:1 (grande); validado em tema claro e escuro.
- i18n: `next-intl` com ICU MessageFormat; message IDs (não string crua); fallback em português.
- Idiomas iniciais: pt-BR e en-US; estrutura pronta para mais sem mudança de código.

14.8 Padrões de frontend Next.js (App Router) otimizado

Esta seção é o resumo executivo da skill `mangashelf-frontend-optimized` (`skills-mangashelf/mangashelf-frontend-optimized/SKILL.md`). A skill é o guardião completo; o que está aqui é o que o reviewer humano valida em todo PR que toca `apps/web/`. O objetivo é impedir os anti-patterns que IA costuma introduzir em Next.js App Router quando escreve React sem entender a diferença para Pages Router.

Regra de ouro: Server Components por padrão. `"use client"` é a exceção, não a regra.

Anti-patterns bloqueantes (não discuta, recuse)

- `useEffect` **para buscar dados.** Use Server Component com `fetch()` + `next: { tags }` para cache + `revalidateTag` para invalidar. TanStack Query APENAS em componentes client que precisam de cache/mutation client-side.
- `"use client"` **no topo de arquivo sem necessidade real.** Se só um botão precisa de interatividade, extraia o botão para um componente client e mantenha o resto como server.
- **Falta de** `Suspense` **para streaming.** Toda `async` function em Server Component tem `loading.tsx` ou `<Suspense>` fallback. Sem isso a página só renderiza quando tudo carrega.
- `<img>` **em vez de** `next/image`**.** Sempre `next/image` com `priority` SÓ na primeira imagem do viewport (LCP).
- `useEffect` **para sincronizar com a URL.** URL é fonte de verdade; use `useSearchParams()` diretamente.
- `useState` **para dados derivados.** Use `useMemo` ou calcule direto na renderização.
- **Falta de** `loading.tsx` / `error.tsx` / `not-found.tsx` **em rotas dinâmicas.** Sem isso o usuário vê tela branca em erro.
- **Falta de Metadata API.** Toda página pública tem `metadata` ou `generateMetadata`. Sem isso não tem SEO.
- **Mutations via** `fetch` **em client quando dá para usar Server Action.** Server Actions com `useFormState` + `useFormStatus`.
- **Cache invalidation manual com** `router.refresh()`**.** Use `revalidateTag` no Server Action para invalidar cache específico.

Quando `useEffect` é a ferramenta certa

- Sincronizar com sistemas externos: WebSocket, EventSource, bibliotecas de UI (carrossel), subscrições a `window.matchMedia`.
- Medidas que só existem no client: `ResizeObserver`, `IntersectionObserver`, `window.scrollY`, `window.innerHeight`.
- Tracking/analytics que não pode ser server-side.
- Atalhos de teclado globais (leitor: ←/→ para trocar página).

Data fetching

- Em Server Components: `fetch()` nativo com `next: { tags, revalidate }`. NUNCA `axios` em Server Component (sem cache nativo).
- Em Client Components (raros): TanStack Query com `staleTime` configurado. NUNCA `fetch` direto sem TanStack Query (sem cache, sem dedup, sem retry).
- Cache invalidation: `revalidateTag('chapter:123')` em Server Action após mutação.
- Streaming: `loading.tsx` em toda rota `async`; `<Suspense>` para blocos que carregam em paralelo; skeletons específicos (não spinner genérico).

Image variants (integra com imgproxy)

Variante	Largura	Uso
thumb	240w	Lista de catálogo
medium	480w	Card de série
mobile-full	800w	Leitor mobile (também web em telas pequenas)
desktop-full	1200w	Leitor desktop
original	original	Download (VIP)

`priority` APENAS na primeira imagem do viewport (LCP). Nunca em todas. `sizes` obrigatório para evitar layout shift e permitir que o browser escolha a variante certa.

Forms e mutações

- Server Actions para mutações. `useFormState` + `useFormStatus` para estado e loading.
- Validação com `zod` no server e `react-hook-form` + `@hookform/resolvers` no client.
- Acessibilidade: `<label>`, `aria-invalid`, `aria-describedby` para erros.
- Progressive enhancement: form funciona sem JS habilitado (Server Action é POST normal).

Roteamento e estrutura

- Pastas para rotas aninhadas.
- `()` para rotas privadas (sem URL): `(auth)/login`, `(admin)/dashboard`.
- `@` para parallel routes (ex.: modal sobre conteúdo).
- `[...catchAll]` para 404 customizado.

- `next/navigation` (NUNCA `next/link`'s `router` para mutações — `router.push` apenas para navegação).

Performance budgets (gate de CI via `mangashelf-perf-budget`)

Métrica	Alvo	Limite (bloqueia)
LCP (Largest Contentful Paint)	< 2.5s em 3G rápido	< 3.0s (bloqueia)
CLS (Cumulative Layout Shift)	< 0.05	< 0.1 (bloqueia)
TBT (Total Blocking Time)	< 200ms	< 400ms (bloqueia)
JS bundle por rota (gzip)	< 200 KB	< 250 KB (bloqueia)
Image weight (total por página)	< 500 KB	< 800 KB (bloqueia)

Roda em CI (Lighthouse nas 5 rotas críticas: home, catálogo, série, capítulo, admin). Bloqueia regressão > 10% em LCP, CLS ou TBT vs branch main. Detalhe na skill `mangashelf-perf-budget`.

14.9 Padrões para app mobile (pré-planejamento)

O app mobile é planejado para depois da Fase 2, mas os padrões abaixo são definidos agora para que o backend já nasça compatível. A skill `mangashelf-mobile-api` (`skills-mangashelf/mangashelf-mobile-api/SKILL.md`) é a referência completa; o que está aqui é o resumo.

Camada	Padrão	Quando
Stack	Expo (managed) + React Native + TypeScript + NativeWind + TanStack Query	Apps mobile futuros
Navegação	Expo Router (file-based, similar ao Next App Router)	Apps mobile futuros
Auth	`expo-secure-store` para refresh token; access token em memória	Apps mobile futuros
Banco local	`expo-sqlite` (SQLite) para progresso offline e cache de capítulos	Apps mobile futuros
Imagens	`expo-image` com `source.uri` apontando para imgproxy; cache automático	Apps mobile futuros
Push	`expo-notifications` abstrai FCM/APNs; server fala FCM/APNs diretamente	Apps mobile futuros
Deep link	`expo-linking` registra scheme `mangashelf://`	Apps mobile futuros
OTA updates	EAS Update para hotfix de JS sem passar pela App Store/Play Store	Apps mobile futuros
Build	EAS Build para iOS (Apple Developer) e Android (Google Play Console)	Apps mobile futuros

Backend já compatível desde a Fase 1: auth com access + refresh rotativo (seção 4), endpoints mobile com prefixo `/api/v1/mobile/`, variantes de imagem `mobile-full` (seção 3.7), ETag em listagens, push notifications via interface `PushProvider`, sync incremental via `/api/v1/mobile/sync`. O custo de fazer isso desde o início é marginal; o custo de retrofit seria alto (quebra de contrato, re-trabalho de auth).

15. Próximos passos imediatos

Antes de começar a Fase 1, deixar tudo abaixo pronto. Ordem sugerida pela dependência técnica:

1. Registrar em ADRs a licença (AGPL-3.0) e o nome (MangaShelf).
2. Criar o repositório com a estrutura da seção 12 e o `.gitignore` adequado.
3. Instalar as skills externas: `clawhub install @dietrichgebert/ponytail` e `clawhub install security-audit` (ou baixar manualmente; ver `skills-mangashelf/README.md`).
4. Copiar o diretório `skills-mangashelf/` para o repo e carregar as skills de contexto (`mangashelf-frontend-optimized` e `mangashelf-mobile-api`) no seu editor (Cursor/Claude Code/Cline/Copilot — ver `skills-mangashelf/README.md`).
5. Configurar pre-commit hooks (Husky + lint-staged + golangci-lint + gitleaks + commitlint) — seção 13.3.
6. Implementar os scripts das skills de gate de CI: `scripts/adr_check.py`, `scripts/migration_check.sh`, `scripts/openapi_diff.sh`, `scripts/perf_compare.js`. Detalhe em cada SKILL.md.
7. Configurar CI com a matriz "mínimo" e "completo", com todos os gates da seção 13.4 + workflows para adr-check, migrations, openapi-diff e perf-budget.
8. Escrever o schema SQL inicial, incluindo o sistema de permissões (seção 4.2), o modelo VIP (seção 5.4) e as tabelas de mobile (`device_push_tokens`, `refresh_tokens`).
9. Definir as interfaces da seção 3.2 (incluindo `PushProvider` e `ImageURLBuilder` com variante `mobile-full`) e o loader de config com validação no boot.
10. Definir o esqueleto do OpenAPI em `api/openapi.yaml` com `/healthz`, `/readyz`, `/api/v1/auth/login`, `/api/v1/auth/refresh`, `/api/v1/mobile/feed`.
11. Configurar branch protection em `main`: PR obrigatório, 2 reviews para `authz/vip/media`/migrations, todos os status checks obrigatórios.
12. Documentar ADR-001 (licença), ADR-002 (stack), ADR-003 (authz atômica), ADR-004 (storage abstraído), ADR-005 (segurança ASVS L2) e ADR-006 (mobile-ready desde o MVP).
13. Criar `docs/runbooks/` com os runbooks da seção 7.8 e o template `cloudflare-audit.md` da seção 7.10.
14. Subir o `docker-compose` de desenvolvimento e validar que `docker compose up` sobe API, worker, web e Postgres vazios.